

Video Transcript

Video 3: Project 1: Creating a Books Web Application (06:52)

In this mini project, we're going to go through a number of tasks related to the books. The first thing that we're going to do in task one is you're going to match up the images of the books so that they can be rendered like this. So, the page, when we go to say, books, the books route would print out the three books with the images like this. So, that's the first task. The second task is we're going to add some new users and we're going to give them roles. So, the roles are going to be either admin or reader. Now, an admin can add a book and add an image, but the reader cannot. So, the reader can look at the books with get books but can't add a book or add an image. So, these are the users, and you can put your own name instead of test user and give yourself a role of admin.

Here, I've got the role of reader and Anna's got the role of admin. So, now we need to arrange that only admin can add books or add images. How are we going to do that? Well, let's take a look at the code. So, in the code that I'm giving you, I've already extended it to include JSON Web Tokens. So, you'll see I'm loading a number of libraries here. You shouldn't need any more libraries than I've actually loaded. And we need to set up the secret key and we're going to store the JSON Web Tokens in cookies. And there are a few other configurations that we're setting. I've added some new books that I like, but you can add your own. There are three of them and I've given them an ID now.

That's because if we look inside static, I've got three images, and I want the image number to match up with the ID of the book. So, that we should be able to figure out which image belongs to which book when we're trying to render it. So, to render it, you're going to need to go into books.html. So, that's what's rendering these books. Here, I've put in the users.

You can alter these and now I've added into where we log in, we're going to set. It's not the session token, it's JWT. And in it we're going to store not only the user's name, but also, some claims. And the one of the claims that we're storing is the role of the user. So, I'm checking the user. So, when they log in, I check their username and password.

I create a valid user that consists of the username and their role. We don't want to store the passwords, so we've got the username and roles. I'm adding the role to the user claims in this JWT token, so that's done here. User claims, I'm sending it equal to and this is a dictionary. So, the name of the claim is role. So, when we're looking for it, that's what we're going to be looking for. And then, when we come to books, we have this decorator JWT required and that's part of the library. We won't be able to see that that function, but what I want you to do is to add now this function as well, which checks if the JSON web token has the role of admin. So, that's what you need to do is you need to write this.

I'm going to give you some clues about how to start it. But what you'll notice here is that we can get the username out of the token, get JWT identity, and there's some other JWT functions that will allow you to pull back that role. You need to look them up and find them. So, your third goal is to, in addition to the JWT required which is given, I want you to write this decorator. Let's go back and see how we might do that. Here, is the outline of `admin_required`. This is how we're calling it. So, we need to complete that decorator. Here's some clues as to how you'd write that. You'd have `admin_required` and you're going to return the wrapper. If it's successful, you return this function.

Otherwise, in here we got to do some kind of test on the role of the user. And remember, we put that role in as a claim into the JSON Web token. So, you're going to need to learn about how this JSON web token library works. So, be careful to look at what library I'm using for the JSON Web Token. It's called `flask_jwt_extended`, `flask_jwt_extended`. That's the library. So, make sure you're looking up the right advice and then you should be able to complete this `admin_required`, this decorator. Okay, and what that will do then, is it'll protect the roots that have that decorator. The user will have to have admin permission to be able to hit those roots. Bye for now.